

BACK_END (SUNDAY 01/03/2026) :

Apps.py: the use of This code in back end is simple and it our Django code can have more info about our app and knows if our site web is running and that's by:

1. Signal Registration - Makes sure your signals (event listeners) are loaded when Django starts
2. App Metadata - Provides Django with info about your app
3. Startup Code - Place to run code when the app initializes

It runs when Django starts and before any requests and after all apps are loaded and this file ensures that our app is properly configured and that all signals are registered when Django starts without crashing if something goes wrong with the signals and it so important so that our app run carefully.

```
from django.apps import AppConfig

class CoreConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "app.core"

    def ready(self):
        # Import signals or perform startup registration here if needed
        try:
            from . import signals # noqa: F401
        except Exception:
            pass
```

Authentication.py: the use of authentication is the process of verifying that someone is who they claim to be and when a user provides credentials like email or password or our system verifies these credentials against stored data and if valid the system creates a session or issues a token and by this token it can subsequently requests so the user doesn't have to log in again so save the log in information and it has 3 cases:

(Evry case with its situation)

Situation	Default JWT	Custom JWT
Request with valid token	☒ Authenticates	☒ Authenticates
Request with invalid token	☒ Raises error	☒ Returns None
Request with NO token	☒ Raises error	☒ Returns None (allows access)

Request arrives

↓

CustomJWTAuthentication.authenticate()

↓

get_header(request) → Check for "Authorization: Bearer <token>"

↓

Is header present?

└─ NO → Return None (anonymous user) ☒

└─ YES → Extract raw token

↓

Is token valid?

└─ NO → Return None (anonymous) ☒

└─ YES → Get user from token

↓

Return (user, validated_token)

And all the informations are sent to the database so that it can be saved and valid for next time log in.

Decorators.py: by these decorators we made 3 general's decorators for our site web :

```

def cache_response(timeout=300, key_prefix="view"):
    """
    Décorateur pour mettre en cache les réponses des vues
    """

    def decorator(func):
        @wraps(func)
        def wrapper(self, request, *args, **kwargs):
            # Construire la clé de cache
            cache_key = (
                f"{key_prefix}:"
                f"{request.path}:"
                f"{request.user.id if request.user.is_authenticated else
'anon'}"
            )

            # Vérifier le cache
            cached_response = cache.get(cache_key)
            if cached_response is not None:
                return Response(cached_response)

            # Exécuter la vue
            response = func(self, request, *args, **kwargs)

            # Mettre en cache si succès
            if response.status_code == 200:
                cache.set(cache_key, response.data, timeout)

            return response

        return wrapper

    return decorator

```

Which caches API responses in Django's cache system to improve performance and it used for :

- Faster responses for repeated requests
- Reduces database load especially for expensive queries
- Better scalability under high traffic

And this is how it works :

Request comes in

↓

Create cache key based on URL + user

↓

Check if response exists in cache

↓

└— YES → Return cached response instantly (no database query!) ⚡

└— NO → Execute the actual view function

↓

Get response from database

↓

If status is 200 (success), save to cache

↓

Return response

```
def require_verified_account(func):  
    """  
    Décorateur pour exiger un compte vérifié (téléphone ou documents)  
    """  
  
    @wraps(func)  
    def wrapper(self, request, *args, **kwargs):  
        if not request.user.is_authenticated:  
            return Response(  
                {"error": "Authentication requise"},  
                status=status.HTTP_401_UNAUTHORIZED,  
            )  
  
        # Vérifier si au moins le téléphone ou un document est vérifié  
        has_verified_phone = request.user.phone_verified  
        has_verified_document =  
request.user.documents.filter(verified=True).exists()  
  
        if not (has_verified_phone or has_verified_document):  
            return Response(  
                {"error": "Veuillez vérifier votre téléphone ou vos documents"},  
                status=status.HTTP_401_UNAUTHORIZED,  
            )  
  
        return func(self, request, *args, **kwargs)
```

```

        {
            "error": (
                "Compte non vérifié. "
                "Veuillez vérifier votre téléphone ou soumettre un
document."
            ),
        },
        status=status.HTTP_403_FORBIDDEN,
    )

    return func(self, request, *args, **kwargs)

return wrapper

```

Ensures users have verified their identity for phone or documents before accessing certain views and it used for :

- Prevents spam from fake accounts
- Builds trust in your community
- Encourages verification for better user quality

And this how it works :

User Status	Phone Verified?	Document Verified?	Access
New user	✗ No	✗ No	🚫 Blocked
Phone verified	✓ Yes	✗ No	✓ Allowed
Document verified	✗ No	✓ Yes	✓ Allowed
Fully verified	✓ Yes	✓ Yes	✓ Allowed

```

def admin_only(func):
    """
    Décorateur pour limiter l'accès aux administrateurs
    """

    @wraps(func)

```

```
def wrapper(self, request, *args, **kwargs):
    if not request.user.is_authenticated or not request.user.is_staff:
        return Response(
            {"error": "Accès réservé aux administrateurs"},
            status=status.HTTP_403_FORBIDDEN,
        )

    return func(self, request, *args, **kwargs)

return wrapper
```

Restricts views to administrators only and admins so for the admins of our site web only and it used for :

- Protects sensitive endpoints
- Reserved for system management
- Separates user/admin functionality

User Type	Authenticated?	is_staff?	Access
Anonymous	✗ No	✗ No	🚫 Blocked
Regular user	☑ Yes	✗ No	🚫 Blocked
Admin	☑ Yes	☑ Yes	☑ Allowed
Superuser	☑ Yes	☑ Yes	☑ Allowed

The summary table :

Decorator	Purpose	When to Use
@cache_response	Speed up repeated requests	Expensive queries, list views, public data
@require_verified_account	Verify user identity	Creating trips, booking, messaging, payments

@admin_only

Restrict to admins

System stats, user management,
moderation tools

And the benefits of this :

- 1.Code reusability - Write once, use on many views
- 2.Separation of concerns - Authentication logic separate from view logic
- 3.Clean code - Views focus on their main purpose
- 4.Consistency - Same rules applied everywhere

These decorators make your API faster (caching), safer (verified accounts), and better organized (admin protection)!